

Software Requirements Specification (SRS)

Inventory Manager

Version: 1.0

Project Type: Python Mini Project

Frontend: HTML/CSS

Backend: Python (Flask)

Storage: JSON File

Document Control

Prepared by: Md. Anjir Jaman

Date: 14-09-2026

Version: 1.0 – Initial draft

Contents

1. Introduction	3
1.1 Purpose	3
1.2 Scope	3
1.3 Intended User	3
2. Overall Description.....	3
2.1 Product Perspective	3
2.2 Main Features	3
2.3 Operating Environment	3
3. Functional Requirements.....	3
4. Data Requirements	4
5. User Interface Requirements.....	4
5.1 Task List Page	4
5.2 Add Task Form	4
5.3 Edit Task Page	5
5.4 Filter Controls.....	5
6. Backend Requirements	5
7. Data Storage Requirements.....	5
8. Non-Functional Requirements.....	5
9. Error Handling	6
10. System Constraints	6
11. Assumptions.....	6
12. Acceptance Criteria.....	6
13. Project Schedule and Milestones.....	6
14. Requirement Summary	7

1. Introduction

1.1 Purpose

The Inventory Manager is a web-based application designed to help users track products, monitor stock movements, search for items, and receive low stock alerts. It provides a simple, lightweight HTML-based interface with a JSON file backend, avoiding the complexity of traditional databases.

1.2 Scope

The system will provide the following main functions:

- Add a product (Name, SKU, Description, Quantity, Low Stock Threshold)
- Record stock movements (Stock In / Stock Out)
- Search for products by name or SKU
- Flag and display low stock items
- Store all inventory records in a local JSON file

1.3 Intended User

Small business owners or individuals needing a simple, local tool to manage stock without complex setups.

2. Overall Description

2.1 Product Perspective

The application will consist of a simple frontend using standard HTML/CSS and a backend powered by Python and Flask. A JSON file will serve as the persistent data storage.

2.2 Main Features

- Add Product
- Record Stock Movement
- Search Products
- View Low Stock Alerts

2.3 Operating Environment

The application will require:

- A computer
- Python 3 with Flask installed
- A modern web browser
- Local file storage

3. Functional Requirements

FR-01: Add Product

The system shall provide a form to add a new product with fields: Name, SKU, Description, Quantity, and Low Stock Threshold.

FR-02: Record Stock Movement

The system shall allow the user to increase (Stock In) or decrease (Stock Out) the quantity of an existing product.

FR-03: Search Products

The system shall provide a search bar to filter the product list by Name or SKU.

FR-04: Flag Low Stock

The system shall compare a product's current quantity against its Low Stock Threshold. If the quantity is equal to or below the threshold, it shall flag the item as "Low Stock" and display an alert.

FR-05: View Product List

The system shall display all products and their current quantities.

FR-06: Data Storage

The system shall save all product data and updates to a local JSON file after every add or stock movement operation.

4. Data Requirements

Each task record shall contain:

Field	Description	Required
ID	Unique task identifier	Yes
SKU	Stock Keeping Unit	Yes
Name	Product name	yes
Description	Optional details	Yes
Quantity	Current stock level	Yes
Threshold	Low stock alert level	Yes

5. User Interface Requirements

5.1 Task List Page

The page shall display all tasks, each showing its title, deadline, priority, and status.

Each task shall provide an Edit link, a Delete button, and a quick status control.

5.2 Add Task Form

The task list page shall include a form with fields for:

- Title
- Description
- Deadline
- Priority

It shall provide an Add Task button.

5.3 Edit Task Page

The page shall display the same fields as the Add Task form, pre-filled with the selected task's current values, plus its Status.

It shall provide a Save Changes button and a link back to the task list.

5.4 Filter Controls

The task list page shall provide a control (such as a dropdown or a set of links) to filter tasks by status.

6. Backend Requirements

The Python (Flask) backend shall:

- Serve the HTML pages
- Process new product submissions
- Process stock movement updates
- Perform search filtering logic
- Identify low stock items
- Read/write data to the JSON storage file

7. Data Storage Requirements

The system shall use JSON for local data storage.

Example inventory.json:

```
[  
  {"id": "1", "sku": "ITEM-001", "name": "Widget A", "description": "Standard widget", "quantity": 15,  
   "threshold": 5}  
]
```

8. Non-Functional Requirements

NFR-01: Usability - Simple, clear, and easy to navigate interface.

NFR-02: Performance - Fast response times for a small to medium number of items.

NFR-03: Maintainability - Clear, well-organized Python code.

NFR-04: Data Persistence - Safely store data in JSON format across sessions

9. Error Handling

The system shall handle common errors without crashing. Messages should be displayed for:

- Missing Name or SKU when adding a product
- Adding a product with a duplicate SKU
- Negative stock quantities during stock out operations
- Missing or corrupted inventory.json file

10. System Constraints

- The backend will be developed using Python and Flask.
- The frontend will use basic HTML/CSS, with no complex JavaScript frameworks.
- Data will be stored locally using JSON.
- No internet connection is required for normal operation.
- No external database is required.

11. Assumptions

The HTML frontend loads successfully.

A product can be added with valid details.

Stock quantities can be correctly updated (In/Out).

The product list can be searched by Name or SKU.

Low stock items are correctly flagged.

Data is saved to the JSON file.

Existing data loads correctly when the application restarts.

Invalid input is handled correctly without crashing.

12. Acceptance Criteria

The HTML frontend loads successfully.

A product can be added with valid details.

Stock quantities can be correctly updated (In/Out).

The product list can be searched by Name or SKU.

Low stock items are correctly flagged.

Data is saved to the JSON file.

Existing data loads correctly when the application restarts.

Invalid input is handled correctly without crashing.

13. Project Schedule and Milestones

Task	9am	10am	11am	12pm	1pm	2pm
Setup project and JSON load/save helpers						
Dashboard UI and View Product List.						
Add Product functionality.						
Stock Movement (Update quantity) functionality.						
Search and Low Stock Alert functionalities.						
Testing and final refinements						

Figure 2: Development schedule

14. Requirement Summary

- FR-01: Add product
- FR-02: Record stock movement
- FR-03: Search products
- FR-04: Flag low stock
- FR-05: View product list
- FR-06: JSON data storage